翁恺课 · 第 5 主题

W5 循环应用与数组初识

对齐翁恺《程序设计入门——C语言》第 5 讲 · 目标：会用循环做批量数据的「求和/求最值/判断」三套路、认识一维数组（一排编号的盒子）并用循环遍历 · 不覆盖：数组与函数、数组运算（W6/W7）

5.1 循环能做的"大事"：把一件小事重复成一批数据

上一讲你会写循环了，但循环到底拿来干嘛？真实程序里，循环最大的用处是**处理一批同类数据**：全班 60 人的成绩、连续一周的温度、一串要排序的数字。翁恺的说法：**循环 + 变量 = 你的"数据流水线"**——一次处理一个，循环把它铺开成一批。

处理"一批"数据，有**三个万用套路**，本讲逐个教：

- **累加器**：一个盒子 sum，不停地往里加 → 求和、求平均、计数。
- **打擂台**：一个变量 max，谁比它大就换人 → 求最大值、最小值。
- **标志变量**：一个开关 isprime，一发现不对就翻脸 → 判断素数、判断"有没有"。

> **一句话：**循环处理批量数据的钥匙是"三个套路"——**累加器、打擂台、标志变量**。这一讲把它们学会，数组只多一件事：把数据**存下来**。

5.2 求平均：累加器（sum）

先看最直白的：读 n 个数，求它们的**总和与平均**。做法是准备一个"累加器" `sum`，每读一个数就加进去：

```
/* wk5_avg.c — 循环读 n 个数求平均值（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    int n, i;
    double sum = 0, x;
    printf("要读几个数？");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        printf("第%d个数：", i);
        scanf("%lf", &x);
        sum += x;
    }
    printf("总和 = %.0f，平均 = %.2f\n", sum, sum / n);
```

```
    return 0;
}
```

输入 3 个数 1、2、3，实跑输出：

```
要读几个数？第1个数：第2个数：第3个数：总和 = 6，平均 = 2.00
```

> **新手坑：累加器忘初始化 = 垃圾值。**如果漏写 `sum = 0`，`sum` 一开始是"不知道多少"的随机数（未初始化），往里面加毫无意义，甚至"碰巧算出对的数"都算错（那是未定义行为）。记住：**求和之前先让 sum = 0**。gcc 在优化开关打开时会给警告，见 5.7 坑①。

5.3 打擂台：求最大值

求一堆数里最大的是谁？别想"一个个记住再比"——内存不够也费劲。翁恺教的老办法：**打擂台**。先随便请第一个数当"擂主"，然后后面每个数依次来挑战，**谁比擂主大，谁就上台**：

```
/* wk5_max.c — 打擂台求最大值（边读边比，不存数组）*/
#include <stdio.h>

int main(void)
{
    int n, i, x, max;
    printf("要读几个数？");
    scanf("%d", &n);

    printf("第1个数：");
    scanf("%d", &x);
    max = x;                    /* 先拿第一个当擂主 */

    for (i = 2; i <= n; i++) {
        printf("第%d个数：", i);
        scanf("%d", &x);
        if (x > max) max = x; /* 有人比擂主大，换人 */
    }
    printf("最大值 = %d\n", max);
    return 0;
}
```

输入 5 个数 3、7、2、9、4，实跑输出：

```
要读几个数？第1个数：第2个数：第3个数：第4个数：第5个数：最大值 = 9
```

> **翁恺提醒：**打擂台的前提是"至少有一个人"。所以 `max` 先取 `a[0]`（或第一个读的数）当擂主，而不是初始化为 0——万一数据全是负数，初始化成 0 就永远找不到最大值了。

5.4 判断素数：标志变量 + break

第三个套路是**标志变量**：一个"开关"，默认没问题（1），一旦发现不对就翻脸（变 0）。经典用例是判断一个数是不是**素数**（只能被 1 和它本身整除）：

```
/* wk5_prime.c — 判断素数（标志变量 + break）*/
#include <stdio.h>

int main(void)
{
    int n, i, isprime = 1;    /* 先假设是素数 */
    printf("输入整数：");
    scanf("%d", &n);

    if (n < 2) {
        isprime = 0;
    } else {
        for (i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                isprime = 0;  /* 被整除了，不是素数 */
                break;        /* 不用再试，退出 */
            }
        }
    }

    if (isprime)
        printf("%d 是素数\n", n);
    else
        printf("%d 不是素数\n", n);
    return 0;
}
```

分别输入 17、15、1，实跑输出：

```
输入整数：17 是素数
输入整数：15 不是素数
输入整数：1 不是素数
```

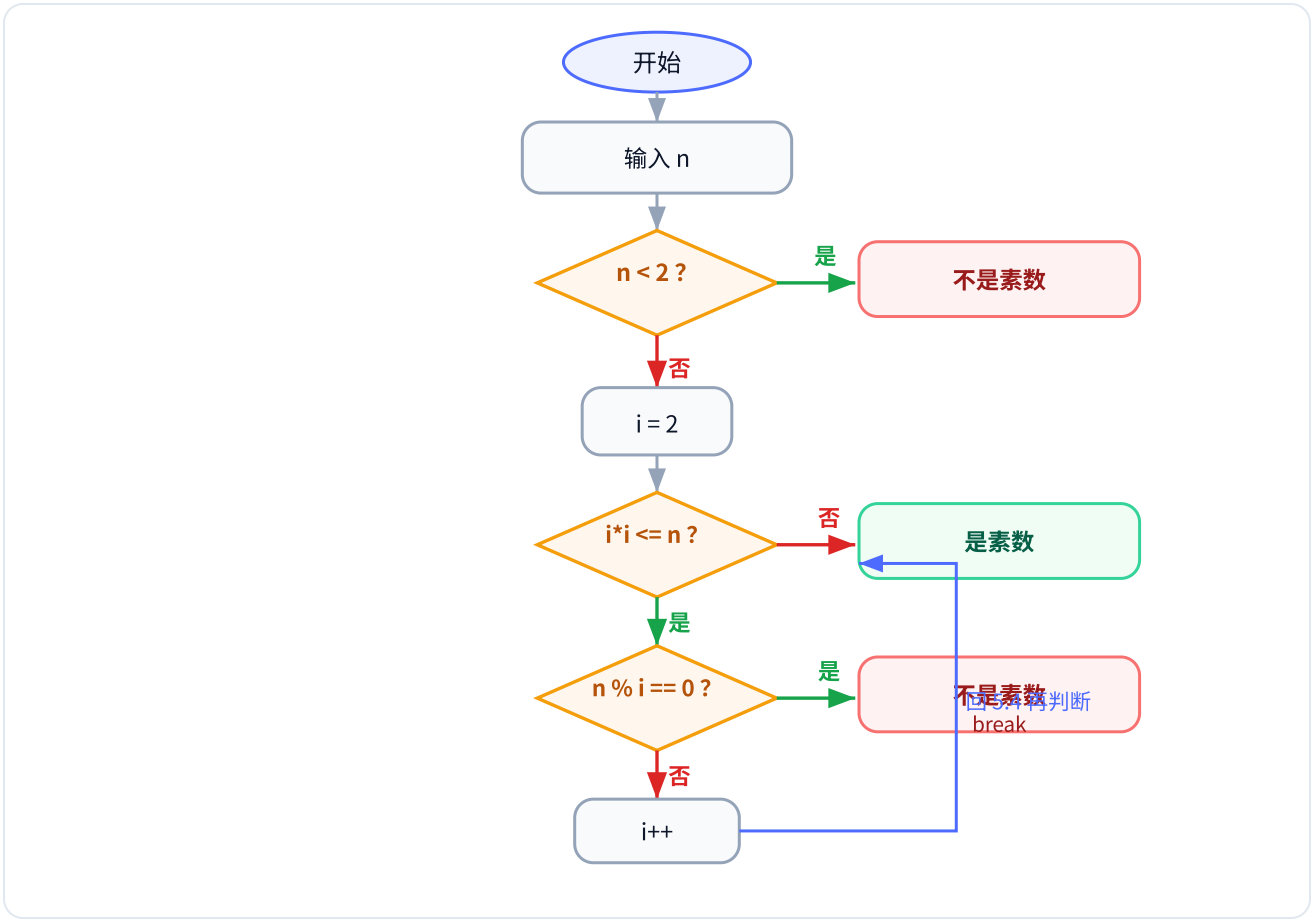


图 W5-2　判断素数流程：先排除 n<2；再从 i=2 试到 $i^2>n$，期间一旦 n%i==0 就置 isprime=0 并 break（不是素数）；试完都没整除就是素数

5.5 数组初识：一排"编号的盒子"

前面三个套路都只用单个变量，但很多问题要**把数据存下来**（比如最后倒序输出、排序、按学号查成绩）。C 用**数组**：连续的一排变量，共用名字，用**下标**区分：

```
int a[5];   /* 声明一个能放 5 个 int 的数组：a[0] a[1] a[2] a[3] a[4] */
```

注意**下标从 0 开始**：第一个是 a[0]，最后一个是 a[n−1]（n 是数组大小）。a[5] 是**越界**——根本没有这个盒子。下面这张图画的就是内存里这 5 个"编号盒子"：



图 W5-1　数组概念图：int a[5] 在内存里是一排连续变量，下标从 0 到 4，a[5] 越界不存在

5.6 数组 + 循环：遍历存取

数组的价值要靠**循环遍历**才发挥出来——一个 for 从 0 数到 n-1，就能挨个访问每个盒子。先看"输入 5 个数再输出"（存下来 = 可以用，这是不存数组做不到的）：

```
/* wk5_arr.c — 一维数组：存 5 个数再遍历输出 */
#include <stdio.h>

int main(void)
{
    int a[5], i;
    printf("输入 5 个整数：");
    for (i = 0; i < 5; i++) {
        scanf("%d", &a[i]);
    }

    printf("你输入的是：");
    for (i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
```

输入 1、2、3、4、5，实跑输出：

```
输入 5 个整数：你输入的是：1 2 3 4 5
```

再配上前面的"打擂台"，就是**数组版求最大值**——先全部存进数组，再从 a[0] 开始打擂台：

```
/* wk5_arrmax.c — 数组版求最大值 */
#include <stdio.h>
```

```c
int main(void)
{
    int a[5], i, max;
    printf("输入 5 个整数：");
    for (i = 0; i < 5; i++) {
        scanf("%d", &a[i]);
    }

    max = a[0];              /* 擂主先取 a[0] */
    for (i = 1; i < 5; i++) {
        if (a[i] > max) max = a[i];
    }
    printf("最大值 = %d\n", max);
    return 0;
}
```

输入 3、7、2、9、4，实跑输出：

```
输入 5 个整数：最大值 = 9
```

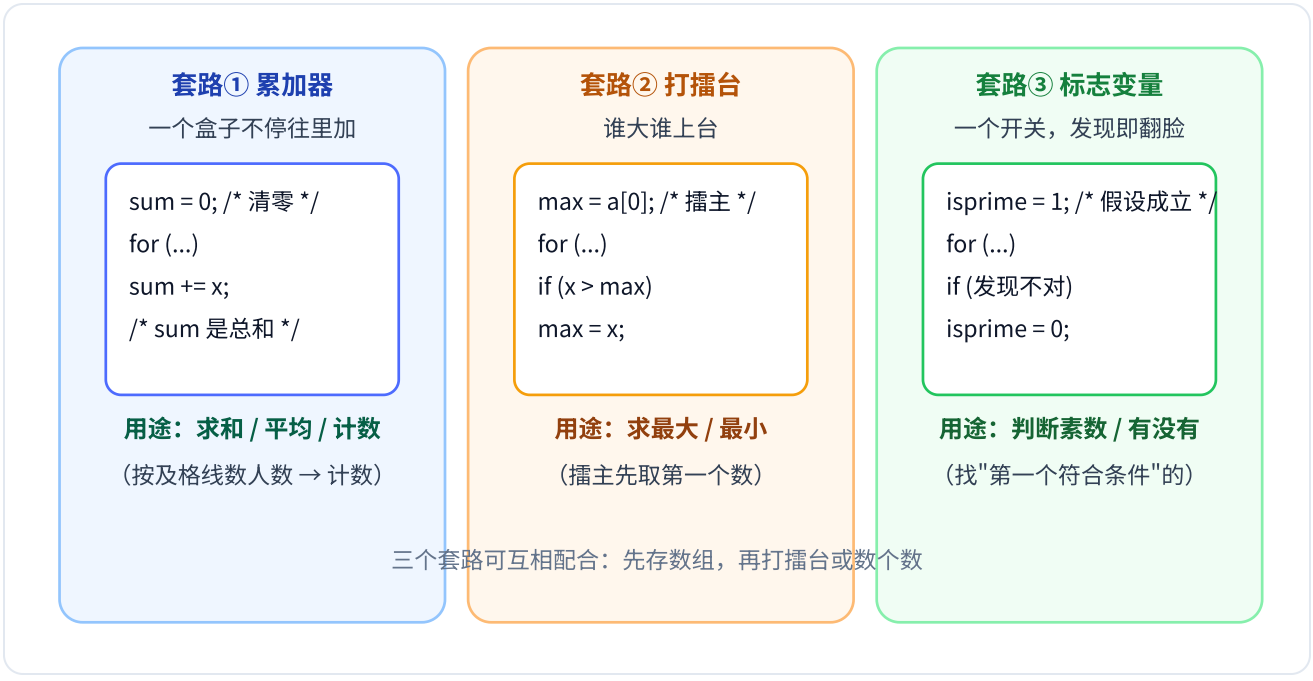


图 W5-3　三种循环套路对比：累加器 / 打擂台 / 标志变量——本讲处理批量数据的万能钥匙

5.7 新手坑集中营（本讲最容易错的 3 件事）

1. **累加器/计数器忘初始化** → 垃圾值或"碰巧对但其实是未定义行为"。修：用之前 `sum = 0;` / `cnt = 0;`。gcc 在 `-O2` 下实测警告：

```
wk5_bug_sum.c:10:13: warning: 'sum' is used uninitialized [-Wuninitialized]
   10 |         sum += a[i];
```

```
    |          ~~~~^~~~~~~
```

> **要点：**这条警告要开优化才冒出来（`-O0` 默认不报），所以初学者常"侥幸跑对"。记住：**编译没警告 ≠ 代码对**，未初始化永远是 bug。

2. **数组越界** → `int a[5]` 只能用 `a[0]~a[4]`，写 `a[5]` 就越界。gcc **经常不报**（未定义行为，编译器不保证拦），运行才可能：读垃圾、写坏相邻变量、或崩溃。遍历循环下标一定写 `i < 数组大小`，别写 `<=`。
3. **下标从 0 开始记错** → 存 5 个数用 `for (i = 0; i < 5; i++)`，不是 `i <= 5`（会越界）也不是 `i = 1; i <= 5`（少用第一个、多用最后一个）。

5.8 一分钟自测（答案见练习册）

判断/填空：

1. 求全班平均分，需要一个变量不停地累加，它应该先初始化为 ____；如果忘了初始化，gcc 在优化时会给哪条警告 ____。
2. `int a[6]` 能用的下标范围是 ____ 到 ____；遍历时循环条件应写成 `i < ____`。
3. 判断素数用"标志变量"：先设 `isprime = ____`，一旦发现 `n % i == 0` 就置 ____ 并 ____（关键字）。

5.9 练习指引

去配套练习册做 W5 练习。翁恺建议本讲里程碑：**写一个"成绩统计"程序**——读 n 个成绩进数组，输出最高分、平均分、不及格人数。这一步把「数组 + 累加器 + 打擂台 + 标志变量」全串起来，是进入 W6 数组运算前最重要的一块拼图。

本讲义为自写原创讲解（对齐翁恺《程序设计入门——C语言》第 5 讲），仅作课堂伴读；示例代码均用 gcc -Wall -Wextra 实测通过（MinGW-W64 gcc 16.1.0），输出为真实运行结果（含 gcc 警告原文）。视频与图片版权归原作者所有，本讲义不含课程原素材。